

# Privacy-Preserving Vision Agent

The full pipeline runs end to end in Chrome and Firefox, against a real local model. What follows is what is genuinely finished, what is not, and the three things worth fixing before we present.

v0.5.4   Chrome MV3 + Firefox MV3   846 unit tests · 56 built-artifact tests · 6 fixtures

|                      |                |                       |                        |
|----------------------|----------------|-----------------------|------------------------|
| 846<br>TESTS PASSING | 0<br>PII LEAKS | ~1s<br>PER AGENT STEP | 33.5 MB<br>PER BROWSER |
|----------------------|----------------|-----------------------|------------------------|

## Where we actually are

A user types a goal in the sidebar, the extension captures the screen, runs a local vision model, detects and redacts PII on-device, sends only the sanitized context to a server running an open-weights model, and executes the single action that comes back. Then it loops until it is done. That whole chain works and has been watched working in a real browser.

Two things are not finished, and one measured result changes how we should talk about the vision model. Those are the sections below.

## Done and verified

Every item here has been observed working in a browser or is covered by a test that fails when the behaviour breaks.

|                                                                                                                                                                                                                                                                                                                                                                                                                          |          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <div>Extension builds and runs on both browsers</div> <div>Manifest V3 on Chrome and Firefox from one codebase. Chrome runs the model in an offscreen document because its background is a service worker with no DOM; Firefox uses its event page directly.</div> <div>56 built-artifact tests assert the emitted manifests, not the config</div>                                                                       | VERIFIED |
| <div>On-device PII detection and redaction</div> <div>Email, phone, credit card, CVV, password and person-name are detected and redacted before anything leaves the machine. Redaction covers both HTML (placeholder, mask, drop-attribute) and pixels (blur, blackout).</div> <div>100% precision and recall across 6 fixtures · 0 leaks · 52 redactions in one real task, 7 of them faces only vision could find</div> | VERIFIED |

## Local vision model loads and runs on WebGPU

VERIFIED

`opencv/face_detection_yunet` is bundled in the extension package and runs through ONNX Runtime directly. 232 KB of weights, 30 ms per frame. Turned off by default behind a panel toggle — see below.

`webgpu backend confirmed in the panel, not assumed`

## Server with a real open-weights model

VERIFIED

Node server speaking the OpenAI chat-completions shape, so it works with Ollama, vLLM, llama.cpp, Together or Groq unchanged. The figures below are the first run against real weights, `qwen2.5:3b` through Ollama on an RTX 4050; the current default is the vision model `qwen2.5vl:3b`.

`237 ms server-side · ~500 ms warm · 3537 MiB VRAM · 46°C`

## Tasks that change the page

VERIFIED

“Search for laptop” types into the field, submits, and reports done in two steps. “Add Laptop Pro to cart” and “Go to the profile” each complete in one. The page genuinely changes — confirmed by the test page’s own console, not just by our panel.

`integration test asserts the live DOM afterwards, not that the step returned ok`

## Bounded agent loop, with eight ways to stop

VERIFIED

done, abort, ask\_user, error, cancelled, no-progress, repeating, and a hard ceiling of 8 steps. Only `done` counts as success — hitting the ceiling means the task did not complete, and the panel says which.

`every exit has its own test`

## The guard that stops a runaway agent

EARNED THE HARD WAY

Asked to add a product to a cart, a model clicked the same button on all eight steps. The cart counter went up each time, so the “did the page change” check kept passing — it reached **10 items** before the step ceiling stopped it. On a real store that is an order. The `repeating` condition now catches the same action on the same element twice running.

worth demoing: it shows the safety design is real, not decorative

## No baked-in endpoints or keys

VERIFIED

The user supplies a server origin at runtime and the browser prompts for permission. No host permissions are granted at install. A test asserts the shipped bytes contain no credentials.

`permissions.request` pinned to a single reviewed file

## The vision model was replaced, and it worked

### yolos-tiny out, OpenCV YuNet in

The old model emitted the 91 COCO classes — person, bicycle, traffic light — of which exactly one mapped to anything we redact. It cost **26 MB and 1765 ms per frame**, over our own latency budget, so it scored zero on that metric while detecting almost nothing.

It is now a purpose-built face detector: **232 KB and 30 ms**. That is 113× smaller and 58× faster, and it detects the thing we actually redact.

Package size dropped from 60 MB to 33.5 MB in the same change.

Verified against real weights: every planted face found at 0.911–0.926 confidence, every control image clean, 32–53 ms, still 4/4 at 1920×1080 where faces are around 73 px.

One honest limit remains. Faces are all vision can reach — `signature`, `id-document` and `credit-card` are covered by regex rules over image alt text and class names, at zero bytes and zero milliseconds, because no model at this budget detects them.

## Not done

### The model has never seen a screenshot

BLOCKING FOR VLM CLAIM

The code path exists and is tested — a redacted screenshot goes as an image part when enabled. It is a panel toggle, defaulting to off, because a vision model plus image tokens does not fit alongside the local model on a 6 GB laptop GPU. So the “VLM” has so far only been used as a text LLM. The local vision model is a separate toggle, also defaulting to off, for the same reason.

one constant to flip, but needs a machine with headroom

### Multi-step tasks beyond two steps

OPEN

The loop supports up to 8 steps, but the on-device baseline deliberately takes one action per goal — a rule added after it sprayed a search query into a payment form’s expiry field and opened five products for a goal naming one. Genuine chaining is the model’s job and has not been demonstrated.

safety rule, not an oversight — but it caps what the baseline can show

### Person names in free prose

KNOWN GAP

A display name sitting in page text is not detected. It needs NER or OCR-plus-classification. This is recorded as a documented gap in the fixtures rather than hidden — it is excluded from the recall denominator and never counted as a false positive.

every excluded item carries a written note; a test enforces that

### Firefox loses the model when its event page unloads

KNOWN GAP

Chrome reconciles this by asking the offscreen document whether a model is loaded. Firefox needs the same treatment; today a step after an unload fails and the user has to press Load model again.

fails safely and says so — it does not silently produce a wrong result

## Worth improving

### Model versus baseline is a trade, not a win

MEASURED

Asked the same goals against the same context: qwen2.5:3b produced a correct select action that the baseline can never emit, and got the checkbox right — but picked the wrong element for “add to cart”, where the word-overlap baseline picked the right one. A 3B model is better at choosing the action type and worse at telling similar elements apart.

a larger model is the obvious next experiment, if a machine can host one

### Vision is off by default

DECIDE BEFORE THE DEMO

With a local model holding 3.5 GB of a 6 GB GPU, the old vision model pushed steps from 1.8 s to over 40 s. It now degrades instead of killing the task, a circuit breaker skips it after repeated failures, and load and inference have separate timeouts. The toggle defaults to off — a decision made when the model was yolos-tiny, and worth revisiting now that YuNet costs 30 ms.

turn it on for the demo and confirm the numbers hold

### Our 100% scorecard is self-graded

BE CAREFUL IN THE DEMO

Five fixtures we wrote, scored by a scorer we wrote. It shows the pipeline is coherent and leak-free on cases we anticipated. It is not evidence about the evaluators’ set, and a straight 100% across every column is itself a sign the fixtures are not adversarial enough.

worth adding harder fixtures before we quote this number

## Package size

WATCH

33.5 MB per browser, down from 60 MB when the vision model was replaced. Almost all of what is left is ONNX Runtime wasm; both builds ship because dropping either removes a working configuration. This feeds metric 4 (client resource use, 20%).

a 64 MB ceiling test exists as a tripwire

## Scorecard as it stands

| FIXTURE          | VISUAL | PII PRECISION | PII RECALL | REDACTION | LEAKS |
|------------------|--------|---------------|------------|-----------|-------|
| benign-docs      | 100%   | 100%          | 100%       | 100%      | 0     |
| checkout         | 100%   | 100%          | 100%       | 100%      | 0     |
| injection        | 100%   | 100%          | 100%       | 100%      | 0     |
| login-form       | 100%   | 100%          | 100%       | 100%      | 0     |
| profile-pii      | 100%   | 100%          | 100%       | 100%      | 0     |
| unlabelled-media | 100%   | 100%          | 100%       | 100%      | 0     |

Read alongside the caveat above: these are our own fixtures. The `injection` one is the interesting case — a hostile page with imperative text, a forged prompt fence and forged redaction tokens, all of which are neutralised.

One caveat the table cannot show: on `checkout` the scorecard reports a sub-threshold detection — a long-digit `bank-account` the system *finds* but never acts on, because it sits below the redaction threshold. It counts as neither a hit nor a leak. It is the one place these fixtures hide a judgement call.

**Do not read this table as real-world recall.** Every fixture is dense with emails, card numbers and phone numbers, which are exactly the kinds that match a pattern. See the real-site run below for what the same pipeline found on a page nobody wrote fixtures for.

## Run it yourself

- `npm install && npm test` — 846 tests, no browser needed.
- `npm run build:chrome` then load `.output/chrome-mv3/` unpacked from `chrome://extensions`.
- Start the planner. Without a model it runs a heuristic baseline and says so on `/health`:

```
npm run ollama:pull
VLM_ENDPOINT=http://localhost:11434/v1/chat/completions \
VLM_MODEL=qwen2.5vl:3b npm run server
```

4. Click the toolbar button **on the page you want to drive** — that click is what grants page access, and it grants it only to that tab.
5. Press **Load model**, grant `http://localhost:8787`, type a goal, press **Run task**.

Try `select India as country` to see the model do something the baseline structurally cannot. The panel names which planner answered, so you can always tell whether you are watching the model or the fallback.

## ■ Off the fixtures: a real site

The pipeline was pointed at a live shopping page of 344 elements — the first time anything here ran against markup nobody on this project wrote. Three things that had only ever been exercised by fixtures held up.

### The context budget did its job

246 elements sent, 98 dropped, ~6975 of 7000 tokens, 9 names truncated — and it *named the refs it dropped* rather than silently shortening the page. Nine real product names exceeded 96 characters, which is the first evidence the truncation limit earns its place.

### YuNet ran on a real page

3 boxes returned from a live viewport. Capture handled 177 KB in 35 ms.

### The model accepted an image

124 elements plus a 131 KB redacted screenshot came back as clean JSON. This is what closes the “no image has ever been sent” gap that earlier drafts of this report listed as the top priority.

The finding that matters most

On that page the DOM scan contributed **almost nothing** — the redactions that fired came from the vision boxes, on a page that should carry an account name.

That is consistent with the oldest known gap here: **person names in free prose are not detected**, and need NER or OCR-plus-classification. A fixture carries emails, card numbers and phone numbers, all of which match patterns. A real logged-in page’s visible PII is frequently just a *name* — precisely the kind this build does not find.

Stated plainly for anyone quoting our numbers: **the fixture suite scores 100% on a class of PII that a real page barely contains.**

not yet confirmed — the session may simply have been logged out; this is the next thing to test

A second attempt returned 688 characters of prose containing no JSON, which the panel had been counting and discarding. Unparseable replies now carry a capped, neutralised snippet of what actually came back — held to the same standard as page content, shown as a diagnostic rather than trusted as one. Two hypotheses (many elements without an image; fewer elements with a 131 KB image) both produced clean JSON, so the cause is something about that site’s content and is still open.

For doing this yourself there is a **plan-only** mode: it runs the whole pipeline against a real logged-in page and withholds the click, so you can inspect what redaction produced without the agent pressing anything.

Against the problem statement

| REQUIREMENT                             | STATUS  | NOTE                                                      |
|-----------------------------------------|---------|-----------------------------------------------------------|
| Extension in Chrome and Firefox         | Done    | Both MV3, one codebase                                    |
| Local vision model via WebGPU           | Done    | YuNet face detector, 232 KB, 30 ms — off by default       |
| Redaction before any network request    | Done    | DOM and pixel; 0 leaks on fixtures                        |
| Server with open-weights LLM/VLM        | Done    | Reached with an image: 124 elements + 131 KB → clean JSON |
| End-to-end task demonstrated            | Done    | Three task shapes complete in a browser                   |
| Visual context transmitted to the model | Partial | Confirmed working; off by default for GPU headroom        |

## ■ If you pick up one thing

Settle whether we detect real PII. On a live page our DOM scan found almost nothing, while our fixtures score 100%. Either the test session was logged out, or the pipeline misses the single most common thing on a real page — a person's name in prose. One logged-in run in plan-only mode answers it. Nothing else on this list changes what we can honestly claim; this does.

Second: turn vision on and re-time a demo run. It was switched off when the model cost 1765 ms a frame. It now costs 30. Nobody has re-measured the decision since the thing it was based on changed.

Third: the screenshot path is proven but still off by default, because the local model and the server model were sharing one 6 GB laptop GPU. Point `VLM_ENDPOINT` at a second machine and both toggles can be on for the demo — that is a deployment choice, not a code change.